- Project 38 — My Learning Book
- Chapter 1 — What is a Distributed System?

Project 38 — My Learning Book

Distributed Systems, learned by building a Parallel LLM Inference pipeline

Student: MD TAUFIQUE HUSSAIN

Built chapter by chapter, together with my teacher. Every chapter explains a concept, shows a picture, and ends with questions I must answer myself.

Chapter 1 — What is a Distributed System?

1.1 The definition

> “A distributed system is a collection of independent computers that appears to its users as a single coherent system.” — Andrew Tanenbaum, *Distributed Systems: Principles and Paradigms*

Two parts matter:

1. **Independent computers** (or processes). Each has its own memory, its own CPU, and its own chance of crashing.
2. **Looks like one system.** The user of Google, WhatsApp or ChatGPT does not care that thousands of machines are involved.

Leslie Lamport (Turing Award winner) described it more bluntly: *“A distributed system is one in which the failure of a computer you didn’t even know existed can render your own computer unusable.”*

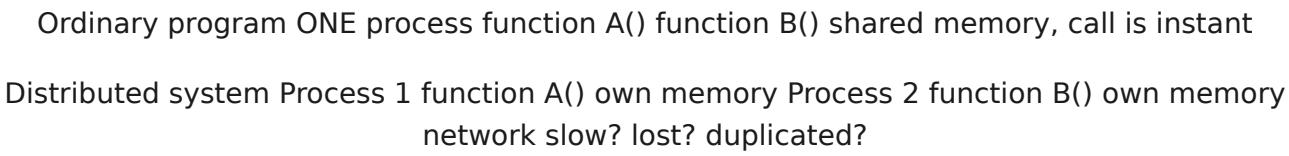


Figure 1.1 — The only difference is the red dashed line: a call that used to be instant and certain is now a message that can be slow, lost or repeated. (Original diagram.)

1.2 Why people build distributed systems

Reason	Example	In Project 38
Size — the job does not fit on one machine	A 70B-parameter model needs ~140 GB; one GPU has ~80 GB	We split GPT-2 into two halves
Speed / throughput — many machines work at once	Search engines	Worker A and B work on different users at once (pipelining)
Fault tolerance — one machine dies, service continues	Banks, WhatsApp	Restart a crashed worker and replay

> **Analogy — the chai shop.** A shop with one person doing everything (take order, boil milk, serve) is a normal program. Now hire a cook and a cashier. Throughput goes up, but new problems appear: the cashier shouts an order and the cook did not hear it (lost message). The cook is slow, so the cashier does not know whether he is busy or fainted (failure detection). The cook makes the same chai twice because the cashier repeated the order (duplicates). *Every one of these is a real distributed-systems problem, and Project 38 solves each of them.*

1.3 Why it is hard: three things you lose

When you move from one process to many, you lose three comforts:

1. **Shared memory.** Process 1 cannot read a variable of Process 2. They can only send messages. (In our project: Worker A cannot look at Worker B's KV cache.)
2. **A shared clock.** Each machine's clock differs slightly, so "who did it first?" is hard. We therefore use *counters* (step numbers, epochs) instead of timestamps.
3. **Reliable calls.** A local function call either returns or the whole program dies. A network call has a **third outcome: you do not know**. The reply might be late, or the other side might be dead, and from outside these look identical.

> **Key idea — partial failure.** In one process, either everything works or everything crashes. In a distributed system, *part* of it can fail while the rest keeps running, and nobody tells you which part. Most of this project (heartbeats, timeouts, epochs, replay) exists to handle partial failure.

1.4 The classic mistakes — the "Eight Fallacies"

Peter Deutsch and colleagues at Sun Microsystems listed false assumptions that beginners make:

1. The network is reliable. 2. Latency is zero. 3. Bandwidth is infinite. 4. The network is secure. 5. Topology does not change. 6. There is one administrator. 7. Transport cost is zero. 8. The network is homogeneous.

Our project attacks #1 (retries, idempotency), #2 and #3 (we *measure* latency and bytes), and #7 (experiment E1).

1.5 Where Project 38 sits

> **Project 38 in one line:** cut GPT-2 into two halves, run each in its own process, connect them with gRPC, and prove that the pair writes *exactly the same text* as the original — even with many users and even when one half is killed mid-sentence.
>
> The ML part is not the point. Every hard part is a distributed-systems problem:

Problem	Concept	Chapter
Halves exchange data	RPC, serialization	5
Each half holds state (KV cache)	Distributed state	3–4
Many users	Concurrency, pipelining	6
Slow stage floods fast stage	Backpressure	7
Crashed vs slow?	Timeouts, heartbeats	8
Crash loses the cache	Recovery by replay	9
Old / duplicate messages	Epochs, idempotency	9

1.6 Think about it (Socratic questions)

> **Q1.** You call a function on another computer to “deduct ₹100 from the account” and get no reply. Do you know whether the ₹100 was deducted? What are the possible situations?
>
> **Q2.** If you just resend the request, what could go wrong? What property would the operation need to make resending safe?
>
> **Q3.** Why can’t two computers simply compare timestamps to decide which event happened first?
>
> *(Write your answers in your own words. In the next chapter I will use them.)*

Chapter 1 summary

- Distributed system = independent processes cooperating through messages, looking like one system.
- You lose shared memory, a shared clock and reliable calls.
- Partial failure and “I don’t know” are the core difficulty.
- Project 38 is a small, safe laboratory for exactly these problems.